

Problem Statement #12 - Healthcare & Telemedicine

Blood Bank Inventory & Emergency Donor Matcher

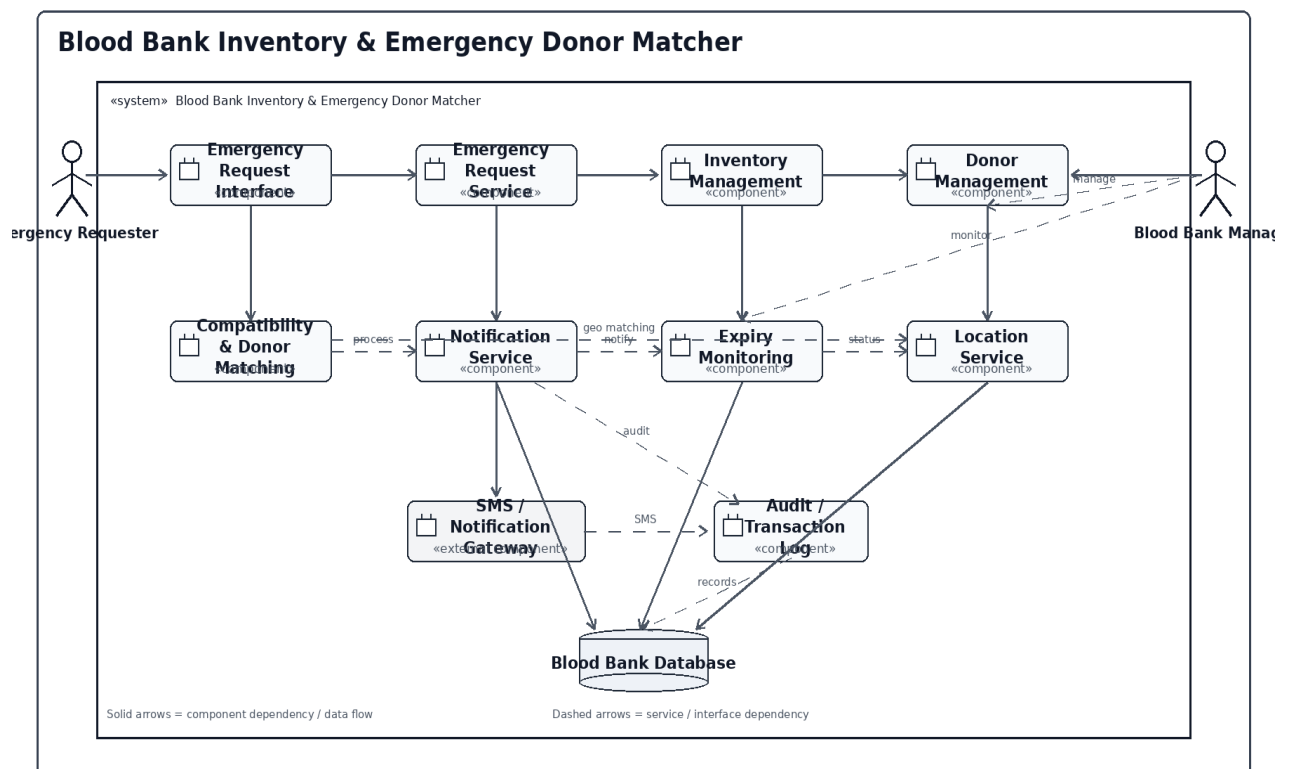
SRN: PES1UG24CS380

Section: Section G

GitHub Repository: <https://github.com/prohan1287/bloodbanksystem>

1. Component Diagram

The component model separates emergency request handling, inventory management, donor management, compatibility matching, notification, expiry monitoring, location support, external notification, and persistent storage



2. Architectural Pattern Selection & Justification

Selected Architectural Pattern:

Layered Architecture

A layered architecture is selected because the system has distinct responsibilities that can be separated into presentation, application/service, business-logic, data-access, and external-service concerns. This keeps emergency request processing, inventory operations, donor matching, and notification workflows organized and maintainable.

Architecture Mapping:

Layer	Responsibilities / Components
Presentation Layer	<i>Emergency Request Interface - receives emergency blood requests and presents system status and results.</i>
Application / Service Layer	<i>Emergency Request Service, Notification Service, and Expiry Monitoring coordinate application workflows.</i>
Domain / Business Logic Layer	<i>Inventory Management, Compatibility & Donor Matching, and Donor Management implement inventory, compatibility, donor eligibility, and matching rules.</i>
Data Access Layer	<i>Handles database operations and transaction processing for blood units, requests, donors, allocations, and expiry records.</i>
External Services	<i>SMS / Notification Gateway and Location Service support emergency communication and geo-targeted donor matching.</i>
Data Store	<i>Blood Bank Database stores persistent inventory, request, donor, and allocation information.</i>

Justification

1. Separation of concerns: User interaction, application workflows, business rules, and persistence are handled separately.
2. Maintainability: Changes to the interface, matching logic, or data storage can be localized to the relevant layer.
3. Data integrity: Inventory operations can be centralized to reduce the risk of allocating the same blood unit to multiple requests.
4. Testability: Matching, donor eligibility, inventory, and expiry rules can be tested independently from the user interface.
5. Extensibility: Additional notification channels or monitoring features can be added with limited impact on other layers.
6. Emergency workflow support: The service layer can coordinate inventory checks, compatibility matching, donor filtering, and notifications in a defined sequence.

Architectural Flow

Presentation -> Application / Service -> Domain / Business Logic -> Data Access
-> Database

n External Services (SMS / Location)